PATTERN RECOGNITION PROJECT REPORT
Fudan University

ZHAO YANG MIN, 14307130067

October 12, 2016

Contents

1 Environment and Organization **2**
1.1 Environment and Language . . . 2
1.2 File Organization . . . 2
1.3 main.py Organization . . . 2
1.4 dataprocess.py Organization . . . 3
1.5 algorithm.py Organization . . . 4

2 Data Processing **7**
2.1 Unbalanced Distribution . . . 7
2.2 Normalization . . . 7

3 Model Evaluation **7**
3.1 Precision, Recall, F1 . . . 7
3.2 Cross Validition . . . 7

4 Artificial Neural Network **8**
4.1 Intuition . . . 8
4.2 Training . . . 8
4.3 Result . . . 8

5 Linear Classification **9**
5.1 Intuition . . . 9
5.2 Projection . . . 9
5.3 Training . . . 9
5.4 Result . . . 10

6 K Nearest Neighbors **10**
6.1 Intuition . . . 10
6.2 Training . . . 10
6.3 Result . . . 11

7 Unbalanced Distribution **11**
7.1 K Nearest Neighbour . . . 11

1 Environment and Organization

1.1 Environment and Language

如果随报告递交的代码有问题，请直接在这个项目的`github`工程中clone代码运行：
https://github.com/yangminz/FDCoursePR

语言：**Python2.7**；主要使用的库：`Numpy`
在安装`Numpy`的`Windows`或`Ubuntu`的`terminal`上通过：

```
python main.py
```

1.2 File Organization

文件的层次组织：

```
./
├──algorithm.py
├──dataprocess.py
├──HTRU_2.csv
└──main.py

0 directories, 4 files
```

其中模型的训练在`main.py`中，数据处理函数写在`dataprocess.py`中，模型算法在`algorithm.py`里。

1.3 main.py Organization

主文件`main.py`的代码结构：

```python
import numpy as np
from algorithm import *
from dataprocess import *

# Artificial Neural Network, K-fold cross validation
def ANN_model(arg_fetr, arg_label, K = 10):
    return [(sum(P) / len(P)), (sum(R) / len(R)), (sum(F1) / len(F1))]

# Logistic Regression, K-fold cross validation
def logRe_model(arg_fetr, arg_label, K = 10):
    return [(sum(P) / len(P)), (sum(R) / len(R)), (sum(F1) / len(F1))]

# K - Nearest Neighbors
def kNN_model(arg_fetr, arg_label, K = 10):
    return [(sum(P) / len(P)), (sum(R) / len(R)), (sum(F1) / len(F1))]

model = {'ANN' : ANN_model, 'logRe' : logRe_model, 'knn' : kNN_model}

def main():
    # sampling('under') - under sample
    # sampling('over') - under sample
    # sampling( 3 ) - 3 unbalanced sample
    case = 0
    if case == 0:
        fetr, label = sampling('over')
```

```
        model['logRe'](fetr, label)
    elif case == 1:
        fetr, label = sampling('over')
        model['logRe'](fetr, label)
    elif case == 2:
        ratio = 8
        judge = []
        for i in xrange(1, ratio):
            fetr, label = sampling(ratio)
            judge.append(model['logRe'](fetr, label))
        print judge

if __name__ == '__main__':
    main()
```

Listing 1: main.py

在`main.py`中定义了三个10折交叉验证的模型训练函数，它们通过

```
        fetr, label = sampling('over')
        model['logRe'](fetr, label)
```

Listing 2: main.py

的方式被采样和调用，并且返还`Precision, Recall, F1`三个模型评价指标。

主函数`main()`提供了模型的选择和采样的方式：

修改`case`值，可以选择下采样、上采样两种平衡的采样方式，以及不平衡的采样方式。设置这三个选项是为了研究数据集中正负样本的分布对训练效果的影响。

修改`model['ANN'](fetr, label)`中的模型名，可以调用不同的模型进行训练。因为在设计函数形参的时候采用了统一的标准，所以可以通过这种方式选择模型。

函数`sampling()`是从`dataprocess.py`中调用来的；三个模型训练函数都是10折交叉验证，其中分别调用了在`algorithm.py`中的模型类。

1.4 dataprocess.py Organization

数据处理文件`dataprocess.py`的代码结构：

```
import numpy as np
import random
import csv

def standScore(arg_M):
    return (arg_M - mean) / std

def readData(arg_csvfile):
    return np.array(positive), np.array(negative)

def unBalRead(arg_csvfile, arg_r):
    return positive, negative[0 : arg_r * len(positive)]

def overSampling(arg_pos, arg_neg):
```

```python
    return data

def underSampling(arg_pos, arg_neg):
    return data

def sampling(case, arg_csvfile = './data/HTRU_2.csv'):
    if case == 'under':
        # Undersampling
        return feature, label
    elif case == 'over':
        # Oversampling and Random Perturbation
    else:
        # unbalanced sampling
        return feature, label

def oneHot(arg_x, arg_dim):
    return vectors
```

Listing 3: dataprocess.py

根据在`main.py`中使用`sampling()`的接口，设计了三种采样模式：下采样、上采样、不平衡采样。前两种是输入字符串形参，不平衡采样是输入负样本对正样本的倍数，因为只是对脉冲星数据集的处理，所以没有反过来设计更精细的情况，比如正样本比负样本多，等等。采样函数返还归一化的特征矩阵`fetr`和对应的标记`label`.

函数`standScore()`是归一化数据处理：输入的矩阵按照列，将均值变成零，方差变成1，这样模型训练时收敛得更快。

`sampling()`中`case=='under'`下采样时，调用函数`undersampling()`，根据少数正样本，随机选取相同数量的负样本，混合打乱，归一化特征以后返回数据集。

`case=='over'`的情况相同，调用函数`oversampling()`，根据多数负样本，随机扰动地生成同样数量的正样本，混合、归一化特征，返还数据集。

不平衡采样的情况调用函数`unBalRead()`，在读取数据集时就按照指定的比率选择正负样本，按照正样本、负样本返还数据集给`sampling()`，然后归一化特征等处理。

单热点函数`oneHot()`是专门为神经网络模型设计的，为了处理标签数据`label`。它将标量标签转换成one-hot向量，方便神经网络的模型训练。

1.5 algorithm.py Organization

模型与算法文件`algorithm.py`的代码结构：

```python
import numpy as np

def sigmoid(arg_M):
    return 1.0 / (1.0 + np.exp(-1 * matr))

def sigmoid2(arg_W, arg_x):
    return 1.0 / (1.0 + np.exp(-1.0 * sum))

def dsigmoid1(arg_M):
    return y * (1 - y)
```

```python

def softmax(arg_M):
    return Exp / Sum[:, np.newaxis]

def CrossEntropy(arg_X, arg_Y):
    return loss / len(arg_Y)

class neuralNet(object):

class logRegress(object):

class kNearest(object):

class ConvolNN(object):
```

Listing 4: algorithm.py

`algorithm.py`中除了数学函数`sigmoid()`, `dsigmoid()`, `softmax()`, `CrossEntropy()`以外，就是四个模型`class`：最基础的单隐层全链接人工神经网络，Logistic Regression的线性分类，K近邻，还有三层的卷积神经网络。其中只实现了前三种，卷积神经网络写起来太麻烦所以暂时没有实现，如果实现了会在github上更新。

```python
class neuralNet(object):
    ''' architecture:
    input - affine - sigmoid - affine - softmax
    '''
    def __init__(self, arg_N, arg_F, arg_H, arg_C = 2):

    def setData(self, arg_X, arg_Y):

    def forward(self, arg_X, arg_Y):
        return z1, h, z2, prob, loss

    def backward(self, z1, h, z2, prob):
        return grad_W1, grad_b1, grad_W2, grad_b2

    def propagate(self):
        return grad_W1, grad_b1, grad_W2, grad_b2, loss

    def modelTrain(self, arg_n):
        return procedure

    def predict(self, arg_X, arg_Y):
        return [P, R, F1]
```

Listing 5: algorithm.py

其中神经网络类的组织如上。其中`init()`初始化模型的参数，`setData`将训练集的特征和对应的标签载入模型。

`forward()`是前向传播的过程，从输入层到隐藏层到输出层。

`backward()`是反向传播过程，计算出两个权重矩阵和两个偏置向量的梯度。其中为了避免梯度爆炸，将梯度的每个元素限制在$[-15.0, 15.0]$之间。

`propagate()`将两个传播过程结合在一起。

`modelTrain()`执行训练过程，根据输入的训练次数进行训练，根据梯度更新参数，返回训练的过程。

`predict()`对测试集进行预测，返还Percision, Recall, F1.

所以，在`main.py`中的模型函数中调用模型训练的做法是这样的：

```
        # model training
        ANN = neuralNet(N, F)
        ANN.setData(train_X, train_Y)
        ANN.modelTrain(2000)
        pred = ANN.predict(test_X, test_Y)
```

Listing 6: main.py

同样，Log Regression的类设计如下：

```
class logRegress(object):
    def __init__(self, arg_N, arg_F):

    def setData(self, arg_X, arg_Y):

    def batchGrad(self, arg_X):
        return grad_W, grad_b, loss

    def modelTrain(self, arg_n):
        return procedure

    def predict(self, arg_X, arg_Y):
        return [P, R, F1]
```

Listing 7: algorithm.py

这里函数的名称和目的基本上和神经网络相同。KNN与这两个模型相比，缺少了模型训练的过程：

```
class kNearest(object):
    def __init__(self, arg_N, arg_F):

    def setData(self, arg_X, arg_Y):

    def cosDist(self, arg_x):
        return np.divide(vecdot, (norm1 * norm2))

    def vote(self, arg_d, arg_k):

    def predict(self, arg_X, arg_Y, arg_k = 100):
        return [P, R, F1]
```

Listing 8: algorithm.py

只需要设置好训练集以后直接做预测就行，在`main.py`中是这样的：

```
        knn = kNearest(N, F)
        knn.setData(train_X, train_Y)
        pred = knn.predict(test_X, test_Y, arg_k = 200)
```

Listing 9: main.py

其中计算样本之间的距离时采用余弦距离。

2 Data Processing

数据预处理主要解决两个问题，一个是正负样本数量不同而带来的过拟合问题，一个是特征值因为度量单位不同而形成的巨大数值差异。

2.1 Unbalanced Distribution

由于现实中发现的脉冲星并不多，所以正样本数据量只有1639，而负样本有16259之多，差不多十倍差距。直接用原始数据训练很容易造成负样本的过拟合，所以可以采用{over, under}-sampling两种方法。

下采样得到的新数据集比较小，所以不全面，预测时可能会出现意料之外的情况。但是它的优点是数据集小，所以训练起来很快。

上采样的数据集比较全面，但是因为正样本重复出现所以可能会出现过拟合的情形。如果胡乱地进行随机扰动，也可能出现“失之毫厘，谬以千里”的惨剧。数据集大，精心设计的随机扰动时间代价也比较高，所以速度慢一点。

不平衡采样的情况是task2要讨论的问题，所以数据预处理也提供了不平衡采样的函数。

2.2 Normalization

特征的数值分布极其不平衡，八个特征之间的变化实在太大，大的数值能到200，小的只有0.2，这样的特征情况对于输入和训练极其不友好，没有归一化时，训练很难收敛。所以需要将数据集归一化：

$$\hat{\mathbf{X}} = \frac{\mathbf{X} - \mu}{\sigma}$$

通过这样的标准归一化方法，能够将每一列的特征数据的均值降为0，而标准差为1，数据变得更为合理，训练难度降低，更容易收敛。

3 Model Evaluation

3.1 Precision, Recall, F1

每次模型的预测函数`model.predict()`都返还这三个结果。

3.2 Cross Validition

交叉验证采用10折交叉验证[3]，这部分融化在`main.py`的模型函数中，以kNN为例，

```
def kNN_model(arg_fetr, arg_label, K = 10):
    foldSize = len(arg_fetr)//K
    P, R, F1 = [], [], []
    for i in xrange(K):
        beg, end = i*foldSize, (i+1)*foldSize
        train_X = np.array(list(arg_fetr[0 : beg]) + (list(arg_fetr[end :])))
        train_Y = np.array(list(arg_label[0 : beg]) + (list(arg_label[end :])))
        test_X = arg_fetr[beg : end]
```

```
        test_Y = arg_label[beg : end]
        # model training
        (N, F) = train_X.shape
        knn = kNearest(N, F)
        knn.setData(train_X, train_Y)
        pred = knn.predict(test_X, test_Y, arg_k = 200)
        if pred != 'cancel this fold':
            P.append(pred[0])
            R.append(pred[1])
            F1.append(pred[2])
    print '+-------------------------------+'
    print '|     10-fold                   |'
    print '|     Precision = %.4f%%        |' % (sum(P) / len(P))
    print '|     Recall =    %.4f%%        |' % (sum(R) / len(R))
    print '|     F1 =        %.4f%%        |' % (sum(F1) / len(F1))
    print '+-------------------------------+'
    return [(sum(P) / len(P)), (sum(R) / len(R)), (sum(F1) / len(F1))]
```

在下采样的基础上，每次取1/10的数据集作为测试集，剩下来的9/10作为训练集，进行10次训练，并且取训练结果的平均值。

在交叉验证中有时候会出现一些奇怪的问题，就是作为P, R的分母: TP+FP, TP+FN 有时候会变成零，这会出现除法错误。这种情况下，判定这一折的训练无效。

4 Artificial Neural Network

4.1 Intuition

简单观察一下数据集，是一个二分类问题。所以第一反应是使用Logisitic线性分类模型，但是数据的分布很可能不能线性划分，所以使用ANN模拟一个复杂的非线性划分，这其中的数学原理请参见*Kolmogorov*在1957年发表的证明*On The Representation of Continuous Functions of Several Variables as Superpositions of Continuous Functions of One Variable and Addition*[2]。

4.2 Training

$$\mathbf{h} = sigmoid(\mathbf{W_1} \cdot \mathbf{X} + \mathbf{b_1})$$

$$\mathbf{Prob} = softmax(\mathbf{W_2} \cdot \mathbf{h} + \mathbf{b_2})$$

在采用undersampling的情况下，一些矩阵和向量的形状如上。

4.3 Result

因为没有暂时配置高效的大数据处理方法，所以采用下采样，并且训练次数较小：每一折只有2000次。这2000 * 10的训练的结果如下：

```
P = 93.7853%, R = 85.5670%, F1 = 89.4879%
P = 93.9394%, R = 92.8144%, F1 = 93.3735%
P = 83.2335%, R = 97.8873%, F1 = 89.9676%
```

```
P = 87.3418%, R = 97.8723%, F1 = 92.3077%
P = 89.3750%, R = 98.6207%, F1 = 93.7705%
P = 91.3580%, R = 97.3684%, F1 = 94.2675%
P = 93.2927%, R = 98.0769%, F1 = 95.6250%
P = 93.5294%, R = 84.5745%, F1 = 88.8268%
P = 85.5346%, R = 97.1429%, F1 = 90.9699%
+-------------------------------+
|     10-fold                   |
|     Precision = 90.1544%      |
|     Recall =    94.4360%      |
|     F1 =        92.0663%      |
+-------------------------------+
```

这个初步实验说明用ANN来处理这个数据集是有效的。这里只采用了单隐层的全连接ANN网络，做了十万次的训练，结果还可以，所以数据集的分布可能不需要比较复杂的函数来逼近。所以采用`log regression`可能其结果也比较好。

要进一步提高性能，可能就要用卷积神经网络了。

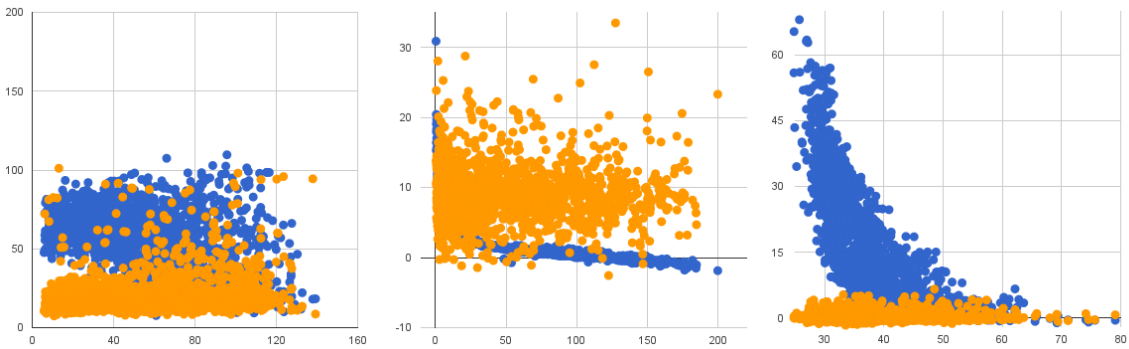
5 Linear Classification

5.1 Intuition

做Logistic线性分类[1]主要是为了测试一下数据的分布是否能够先行划分。我们知道，由于Kolmogorov定理，神经网络模型能够逼近复杂的连续函数，从而给出恰当的划分，这在上一节中的训练里表现了出来。但是数据集能否良好地线性划分呢，这是一个比较有趣的问题，所以采用Logistic回归。

5.2 Projection

先取了几组投影观察一下数据的分布情况：



从投影的结果可以清晰地看到，正负样本之间聚类的情况比较好，基本上也能找到划分的界线，所以估计Logistic回归的效果可能不错。

5.3 Training

用sigmoid函数做二分类：

$$\frac{1}{1+e^{\mathbf{W}\cdot\mathbf{x}+\mathbf{b}}}$$

5.4 Result

因为Log Regression模型的计算量远远小于神经网络，所以下采样下交叉验证时每折五万次，进行十次交叉验证，其结果为：

```
P = 89.8810%, R = 98.6928%, F1 = 94.0810%
P = 89.1720%, R = 98.5915%, F1 = 93.6455%
P = 88.3598%, R = 97.6608%, F1 = 92.7778%
P = 92.5466%, R = 94.9045%, F1 = 93.7107%
P = 94.5783%, R = 97.5155%, F1 = 96.0245%
P = 91.3514%, R = 93.8889%, F1 = 92.6027%
P = 87.7698%, R = 93.1298%, F1 = 90.3704%
P = 88.9655%, R = 96.2687%, F1 = 92.4731%
P = 92.9412%, R = 75.5981%, F1 = 83.3773%
P = 90.3226%, R = 92.1053%, F1 = 91.2052%
+-------------------------------------+
|     10-fold                         |
|     Precision = 90.5888%            |
|     Recall =    93.8356%            |
|     F1 =        92.0268%            |
+-------------------------------------+
```

整体看性能还是可以的，和神经网络的差距不大。但是某些训练结果比较突出，而且考虑到计算量、计算速度、实现的难度，线性分类还是首选。

如果问题本身就是一个线性问题，那么迂回地用神经网络去逼近的效果当然没有直接做线性分类的效果好。但是两组试验的训练次数也存在比较的差异，虽然如此，线性回归的速度依然快于神经网络，如果神经网络能分布式计算较大的训练次数，或许效果会更好。

6 K Nearest Neighbors

6.1 Intuition

之前在线性分类里通过投影看到正负样本之间聚集的情况比较好，所以使用KNN的效果应该很好。

6.2 Training

最初实验的时候，采用了切比雪夫距离：

$$||\mathbf{x}-\mathbf{y}|| = \left(\sum_{i=1}^{dim}(x_i - y_i)^p\right)^{\frac{1}{p}}$$

尝试了几个p值，比如欧几里得值，但是效果基本上不超过30%。

转而实验余弦距离：

$$||\mathbf{x}-\mathbf{y}|| = \frac{<\mathbf{x},\mathbf{y}>}{\left(\sum_{i=1}^{dim} x_i^2 \cdot \sum_{i=1}^{dim} y_i^2\right)^{\frac{1}{2}}}$$

K取50，效果出奇的好，而且计算时间还要短于线性回归。

6.3 Result

```
P = 98.7097%, R = 100.0000%, F1 = 99.3506%
P = 97.4576%, R = 100.0000%, F1 = 98.7124%
P = 97.1429%, R = 100.0000%, F1 = 98.5507%
P = 98.5401%, R = 99.2647%, F1 = 98.9011%
P = 97.1631%, R = 99.2754%, F1 = 98.2079%
P = 95.1389%, R = 100.0000%, F1 = 97.5089%
P = 97.9866%, R = 99.3197%, F1 = 98.6486%
P = 97.3684%, R = 99.3289%, F1 = 98.3389%
P = 94.5578%, R = 100.0000%, F1 = 97.2028%
P = 99.2754%, R = 99.2754%, F1 = 99.2754%
+-----------------------------------+
|     10-fold                       |
|     Precision = 97.3341%          |
|     Recall =    99.6464%          |
|     F1 =        98.4697%          |
+-----------------------------------+
```

具体原因搞不清楚。

7 Unbalanced Distribution

考虑正负样本分布不均的影响。

7.1 K Nearest Neighbour

KNN的情形，为了提高速度将投票的候选人数量降到21：

```
1:1
+-----------------------------------+
|     10-fold                       |
|     Precision = 96.9005%          |
|     Recall =    99.3006%          |
|     F1 =        98.0830%          |
+-----------------------------------+
1:2
+-----------------------------------+
|     10-fold                       |
|     Precision = 96.2008%          |
|     Recall =    99.4404%          |
|     F1 =        97.7861%          |
+-----------------------------------+
1:3
+-----------------------------------+
|     10-fold                       |
|     Precision = 96.6734%          |
|     Recall =    99.2884%          |
|     F1 =        97.9580%          |
+-----------------------------------+
1:4
+-----------------------------------+
|     10-fold                       |
|     Precision = 95.4427%          |
|     Recall =    99.5853%          |
|     F1 =        97.4599%          |
```

```
+-----------------------------------+
1:5
+-----------------------------------+
|     10-fold                       |
|     Precision = 95.5509%          |
|     Recall =    99.3391%          |
|     F1 =        97.4008%          |
+-----------------------------------+
1:6
+-----------------------------------+
|     10-fold                       |
|     Precision = 95.0897%          |
|     Recall =    99.2942%          |
|     F1 =        97.1408%          |
+-----------------------------------+
1:7
+-----------------------------------+
|     10-fold                       |
|     Precision = 93.9345%          |
|     Recall =    99.5151%          |
|     F1 =        96.6336%          |
+-----------------------------------+
1:8
+-----------------------------------+
|     10-fold                       |
|     Precision = 94.6546%          |
|     Recall =    99.1884%          |
|     F1 =        96.8590%          |
+-----------------------------------+
```

可以看到三个模型评价指标没有太大的变化。我感觉这个结果对于KNN模型而言是合理的，因为既然KNN模型没有训练参数，只有计算距离，所以它的预测完全依赖于样本的空间分布，而与样本分布是否平衡无关。打个比方，就像银河中的太阳系，无论太阳系外的天体数量如何多，如果样本天体落在太阳系内，总是可以计算到它距离太阳最近，而与系外天体无关，所以不会产生过拟合的问题。

也可以看到负样本数量压倒性地多的时候，确实对模型产生了轻微的影响。毕竟负样本数量多，混入正样本空间的样本点也变多了，这对投票会产生影响，但是并不显著。

但是基于训练的模型却并不如此，它们应该要受训练样本分布的影响：

```
1:1
+-----------------------------------+
|     10-fold                       |
|     Precision = 85.0957%          |
|     Recall =    93.8479%          |
|     F1 =        89.0138%          |
+-----------------------------------+
1:2
+-----------------------------------+
|     10-fold                       |
|     Precision = 89.9788%          |
|     Recall =    91.6714%          |
|     F1 =        90.7429%          |
+-----------------------------------+
1:3
+-----------------------------------+
|     10-fold                       |
```

```
|     Precision = 88.4139%          |
|     Recall =    92.3190%          |
|     F1 =        90.2852%          |
+-----------------------------------+
1:4
+-----------------------------------+
|     10-fold                       |
|     Precision = 88.4975%          |
|     Recall =    88.7974%          |
|     F1 =        88.4992%          |
+-----------------------------------+
1:5
+-----------------------------------+
|     10-fold                       |
|     Precision = 85.9963%          |
|     Recall =    89.0196%          |
|     F1 =        87.2397%          |
+-----------------------------------+
1:6
+-----------------------------------+
|     10-fold                       |
|     Precision = 88.4643%          |
|     Recall =    74.7816%          |
|     F1 =        80.0384%          |
+-----------------------------------+
1:7
+-----------------------------------+
|     10-fold                       |
|     Precision = 85.7485%          |
|     Recall =    85.5939%          |
|     F1 =        85.6050%          |
+-----------------------------------+
1:8
+-----------------------------------+
|     10-fold                       |
|     Precision = 85.0488%          |
|     Recall =    77.6573%          |
|     F1 =        79.5264%          |
+-----------------------------------+
```

总体上性能还是在下降的。这是实际上，在做10折交叉验证的时候，选取的测试集是本身也是不平衡的，所以在这里准确率下降的不明显。如果单独测试一组测试集，就会看出这一点了。

References

[1] Machine Learning in Action. *Machine Learning in Action*. Manning Publications Co., 2012. ISBN: 9781617290183.

[2] A. N. Kolmogorov. *Selected Works of A.N.Kolmogorov Volume I Mathematics and Mechanics*. Kluwer Academic Publishers, 1991. ISBN: 9027727961.

[3] Zhi-Hua Zhou. *Machine Learning*. Tsinghua University Press, 2016. ISBN: 9787302423287.